

Exercise Sheet 3

Lab Sessions: May 7/15, 2026

In Exercise Sheet 2, you made your experiment *repeatable*. Now, we take the next step: a gold-standard *reproducibility* package, where the **entire analysis can be executed with a single command**.

1 Preparation

This exercise sheet continues exercises from Sheet 2, for which we provide input files and skeleton code through the [RepEng FIMGit Repository](#). Update your local repository that you cloned in the first lab session:

1. Open a terminal in your local repository.
2. Pull the latest changes using `git pull`

2 Experiment Dispatching

We now extend the existing `run_experiment.sh` script from Exercise Sheet 2 into a central dispatcher for running all experiments.

2.1 Extending `run_experiment.sh`

Modify the existing `run_experiment.sh` script from Exercise Sheet 2 step by step. The current script already accepts an input file and the number of runs. Extend the script so that it also supports a random seed and an experiment label. The intention is that users can label their experiments with a string of their choice. This string becomes part of the path where the experiment results are stored. Additionally, the result files should be stored in a `result` directory instead of a temporary one.

For example, below the Shell script is called with input file `recipe.txt`, 10 runs, a seed of 42, and a label `exp1`:

```
./run_experiment.sh recipe.txt 10 42 exp1
```

- (a) **Random seed argument.** Add a third positional argument for the random seed. In `run_experiment.sh`, pass the seed to `pplease.py` so that the polite text is generated deterministically.

- (b) **Result directory and label.** Add a fourth positional argument for the experiment label. Instead of storing intermediate files in a temporary directory, store the generated outputs in a persistent `results/` directory. The results for each experiment should be stored in a dedicated subdirectory in `results/` and all parameters should be encoded in the subdirectory name.

For example, for the input file `recipe.txt`, 10 runs, seed 42, and label `exp1`, the result directory should be `results/recipe_runs_10_seed_42_exp1/`.

- (c) **Output files per run.** Change the script so that each run creates one output file in the experiment result directory instead of a temporary directory. For example, the first run should create the file `polite_1.txt`. Change the path of the statistics output to the results directory as well.

- (d) **CSV file with generated strings.** In addition to the text output files, store the strings in a CSV file named `results.csv` inside the result directory. The CSV file should contain one row per run and sentence and have the columns *run*, *original*, and *polite*.

For example, if a sentence “400g white sugar.” is changed to “400g white sugar, please.” in the second run, the following row is added to the CSV file:

```
2,"400g white sugar.,""400g white sugar, please."
```

- (e) **Usage block.** The usage block in a script is an if-condition, which checks if the script was called properly from the command line. The usage block in this script only checks the correct number of parameters. If this check fails, the correct syntax of calling this script from the command line is printed. Adapt the existing usage block (Lines 10–13 in `run_experiment.sh`) to the new command-line syntax.

2.2 Adapting the Analysis

Modify your `pplease_stats.py` script so that it reads the generated CSV file instead of comparing individual text files.

The script should now be called with two arguments, (i) the path to the CSV file, (ii) the run number to analyze.

For example, if the first run should be analyzed, we call this as follows:

```
python3 pplease_stats.py results.csv 1
```

The script should read the CSV file, select only the rows belonging to the specified run, and compute the minimum, maximum, and median string length for the columns *original* and *polite*.

2.3 Adapting the Dockerfile

Adapt the Dockerfile that you completed in Exercise Sheet 2 so that it runs the full experiment workflow with the updated `run_experiment.sh` script.

The container should execute the experiment with the input file `recipe.txt`, 10 runs, random seed 42, and label `docker`. When the container is run, it should create the corresponding result directory and print the experiment output.

2.4 Creating a doAll.sh Script

Create a new script `doAll.sh` that runs the complete experiment with the following fixed parameters: 100 runs, random seed 42, label `doAll`.

The script should be executable inside the container and run the experiment for the input file `recipe.txt`.

3 Binary Container

So far, we focused on *source containers* (rebuilding from a Dockerfile).

Now, we consider **binary containers**, i.e., containers built from a Docker image where **all dependencies are fixed in time**.

- Build an image from the Dockerfile modified in Exercise 2 (replace `<matr_nr>` with your matriculation number):

```
docker build -t lab3_<matr_nr> .
```

- Export the image:

```
docker save -o lab3_<matr_nr>.tar lab3_<matr_nr>
```

- Exchange the image (the created tar file) with that of a fellow student. Load the other student's image (assume the filename is `lab3_<other_matr_nr>.tar`):

```
docker load -i lab3_<other_matr_nr>.tar
```

- Run the fellow student's binary container:

```
docker run --rm lab3_<other_matr_nr>
```

- Compare the output with your output and provide feedback to the fellow student. Receive feedback from the fellow student gracefully.

4 Dependency Management (Antwort-Wahl-Verfahren)

In this exercise, each question has exactly one correct answer. Mark the correct answer.

Consider the Dockerfile and the Python script `experiment.py`.

Dockerfile

```
1 # Builds the environment for our text analysis
2 # Copyright 2026, John Smith
3 # SPDX-License-Identifier: MIT
4 FROM ubuntu:latest
5
6 LABEL org.opencontainers.image.authors="john.smith@example.com"
7
8 RUN apt-get update && apt-get install -y \
9     ca-certificates \
10    curl \
11    python3=3.14.3-0ubuntu2 \
12    python3-certifi=2026.1.4+ds-1 \
13    python3-jschema=4.19.2-6ubuntu2 \
14    python3-requests=2.32.5+dfsg-1ubuntu1
15
16 WORKDIR /app
17 COPY experiment.py /app/
18 CMD ["python3", "/app/experiment.py"]
```

experiment.py

```
1 import requests
2 from jsonschema import ValidationError, validate
3
4 # Fetch the current UTC time from an external service
5 # and validate that the response has the expected JSON structure:
6 # the properties year, month, day, hour, minute, and seconds
7 # must be integers. This is done by validating against a JSON Schema.
8 SCHEMA = {
9     "type": "object",
10    "required": ["year", "month", "day", "hour", "minute", "seconds"],
11    "properties": {
12        "year": {"type": "integer"}, "month": {"type": "integer"},
13        "day": {"type": "integer"}, "hour": {"type": "integer"},
14        "minute": {"type": "integer"}, "seconds": {"type": "integer"}
15    }
16 }
17
18 response = requests.get(
19     "https://timeapi.io/api/Time/current/zone?timeZone=UTC")
20
21 data = response.json()
22 try:
23     validate(instance=data, schema=SCHEMA)
24     print("Time data valid against required schema.")
25 except ValidationError as exc:
26     print(f"Time data invalid against required schema: {exc.message}")
```

You save the image from the provided **Dockerfile** and share all artifacts with a fellow student. After two months, your colleague (i) loads and runs the **binary container**, (ii) builds and runs the **source container** from the **Dockerfile**.

Which statements are correct?

- (a) If Ubuntu has a new release, the reproduction of the experiment may fail in...
 - ☐ the binary and the source container
 - ☐ only the binary container
 - ☐ only the source container
 - ☐ neither the binary nor the source container
- (b) If a new Python version is released, the reproduction of the experiment may fail in...
 - ☐ the binary and the source container
 - ☐ only the binary container
 - ☐ only the source container
 - ☐ neither the binary nor the source container
- (c) If the external time service used by `experiment.py` is offline, the reproduction of the experiment may fail in...
 - ☐ the binary and the source container
 - ☐ only the binary container
 - ☐ only the source container
 - ☐ neither the binary nor the source container